

MINI PROJECT PRESENTATION

IoT-Based Patient Monitoring Scenario

An AIoT Pipeline for Health Data Management:
From Acquisition to Anomaly Detection

Presented By:

Manish Kumar Tiwari

Research Student, MAS-Data Science

Roll No: 025/MDS/03 | 2025 Batch

Presented To:

Asst. Prof. Dr. Rituraj Lamsal

Department of Digital Technology

Madan Bhandari University of Science and Technology

2026-03-27

Project Objective

Continuous monitoring of vital signs outside of clinical settings produces significant amounts of data, making it suitable for early warning and patient health management.

Practical Applicability

Real-world Ready: Pipeline can be adapted for actual IoMT devices

Scalable: Handles multiple patients and extended monitoring periods

Cost-effective: Synthetic data eliminates privacy concerns

Deployable: Compatible with Google Colab and Anaconda environments

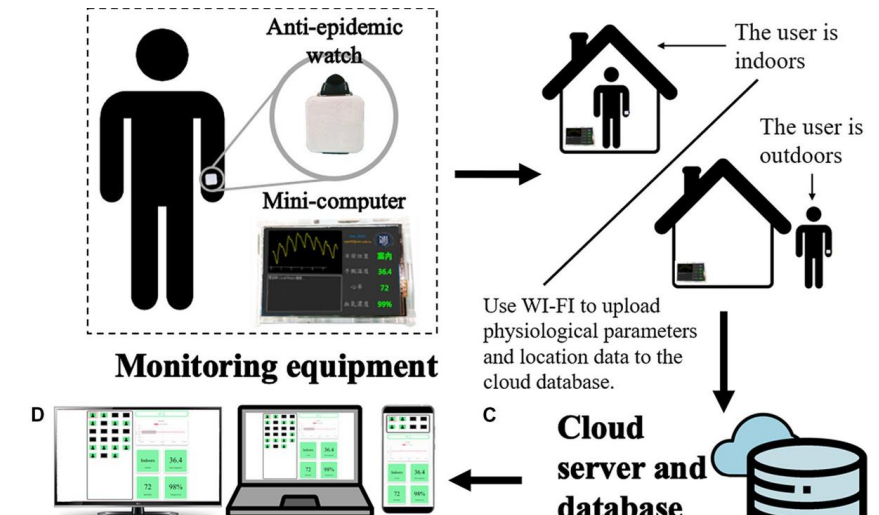
01 Project Overview

Internet of Medical Things (IoMT)

IoMT devices collect and record vital signs such as heart rate, blood oxygen levels (SpO2), and body temperature in real-time. Continuous monitoring of vital signs outside clinical settings produces significant amounts of data, making it suitable for early warning systems and patient health management.

Three-Stage AIoT Pipeline

- 1 Data Acquisition**
Synthetic IoT sensor data generation
- 2 Data Cleaning**
Handling missing data, outliers, and noise
- 3 Processing & Anomaly Detection**
Feature extraction and ML-based detection



IoT-based wearable health monitoring device architecture

The Challenge

- **Real patient data is often inaccessible** due to privacy regulations (HIPAA, GDPR) and institutional restrictions
- **Limited availability of labeled anomaly data** for training machine learning models
- **Need for continuous monitoring systems** that can operate outside hospital environments
- **Early detection of health deterioration** requires intelligent anomaly detection algorithms

The Solution

Synthetic data generation provides a practical alternative when access to real patient data is limited. This approach:

- ✓ Generates statistically similar data to real-world physiological signals
- ✓ Allows controlled injection of anomalies for model training
- ✓ Enables reproducible research and algorithm validation
- ✓ Eliminates privacy and ethical concerns

Target Vital Signs

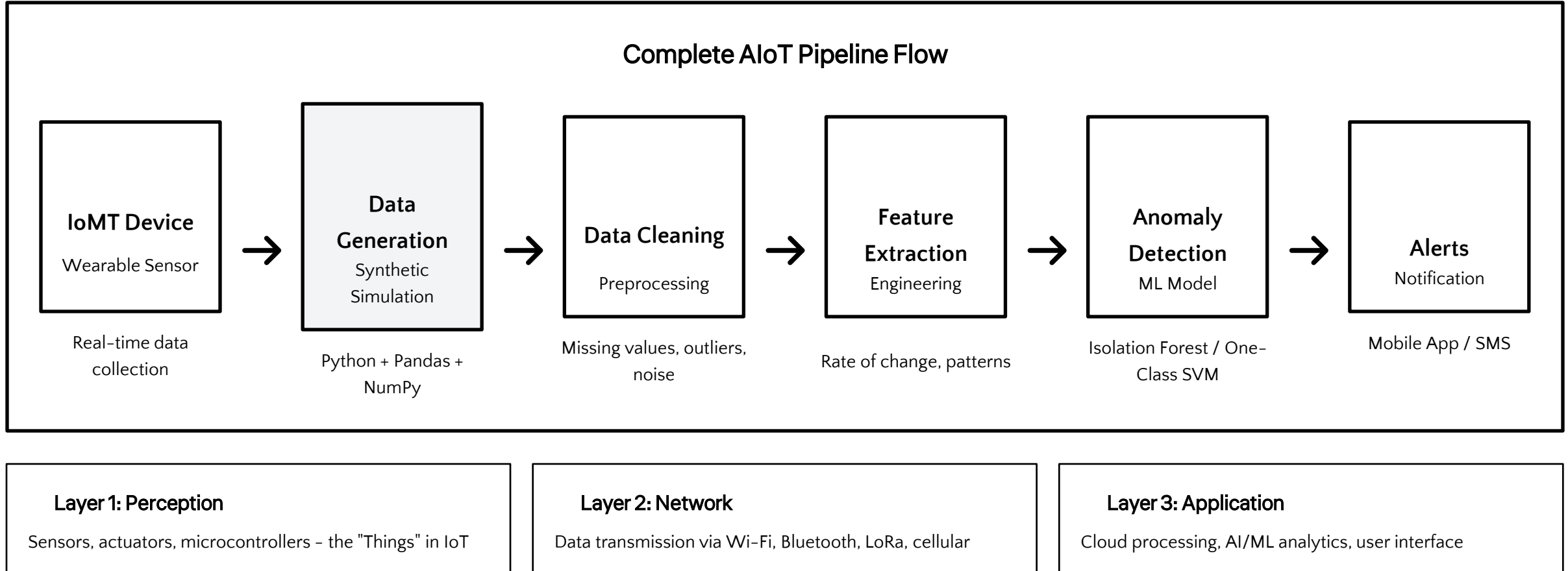
Heart Rate
60-100 bpm

SpO2
95-100%

Temperature
97-99°F

Project Scope

- **Patients:** 5 simulated patients
- **Duration:** 24 hours (1,440 minutes)
- **Frequency:** 1 reading per minute
- **Total Data Points:** 7,200 records



Synthetic IoT Sensor Data Generation

Baseline Generation

Normal physiological signals are generated with **sinusoidal trends and Gaussian noise** to simulate realistic daily variations:

Heart Rate (HR)

60–100 bpm baseline with ± 10 bpm variation

Blood Oxygen (SpO2)

95–100% baseline with $\pm 0.5\%$ variation

Temperature

97–99°F (36.1–37.2°C) baseline with $\pm 0.2^\circ\text{C}$ variation

Anomaly Injection

Realistic problems are introduced to simulate sensor errors and patient health issues:

Sudden Spikes/Drops

± 30 bpm HR changes, ± 5 –10% SpO2 drops, ± 0.5 –1.0°C temp changes lasting 5 minutes

Extreme Outliers

HR = 200 bpm, SpO2 = 50%, Temperature = 40°C or $< 35^\circ\text{C}$

Missing Values

Random 10 missing values per patient (transmission errors)

Data Structure

DataFrame columns:
[timestamp, patient_id,
heart_rate, spo2,
temperature, is_anomaly]

Anomaly Labeling Criteria

- **HR Anomaly:** > 150 or < 30 bpm
- **SpO2 Anomaly:** $< 88\%$
- **Temp Anomaly:** $> 39^\circ\text{C}$ or $< 35^\circ\text{C}$
- **Missing Data:** Any null value

Synthetic IoT Health Data Simulation

```
import pandas as pd
import numpy as np
```

Configuration

```
patients = range(1, 6) # 5 patients
time_index = pd.date_range("2024-01-01",
periods=1440, freq='min')
```

Generate baseline data with variations

```
for pid in patients:
    t = np.linspace(0, 2*np.pi, len(time_index))
    hr = 75 + 10*np.sin(t) + np.random.normal(0, 2,
len(time_index))
    spo2 = 97 + 0.5*np.sin(t*2) + np.random.normal(0, 0.2,
len(time_index))
    temp = 37 + 0.2*np.sin(t*3) + np.random.normal(0, 0.1,
len(time_index))
```

Inject anomalies

```
for _ in range(5):
    idx = np.random.randint(50, len(time_index)-5)
    hr[idx:idx+5] += np.random.choice([30, -25])
    spo2[idx:idx+5] -= np.random.choice([5, 10])
    temp[idx:idx+5] += np.random.choice([0.5, 1.0])
```

Key Libraries

- ✓ **Pandas:** Data manipulation
- ✓ **NumPy:** Numerical operations

Data Characteristics

- **Temporal:** Time-series data
- **Multi-variate:** 3 vital signs
- **Multi-patient:** 5 subjects
- **High-frequency:** 1 min intervals
- **Realistic:** Physiological patterns
- **Challenging:** Mixed anomalies

Add extreme outliers

```
hr[np.random.randint(len(time_index))] = 200  
spo2[np.random.randint(len(time_index))] = 50  
temp[np.random.randint(len(time_index))] = 40
```

Introduce missing values

```
missing = np.random.choice(len(time_index), size=10,  
replace=False)  
hr[missing] = np.nan  
spo2[missing] = np.nan  
temp[missing] = np.nan
```

Sample Output

timestamp	patient_id	hr	spo2	temp	is_anomaly
-----------	------------	----	------	------	------------

2024-01-01 00:00:00	1	75.2	97.1	37.0	0
---------------------	---	------	------	------	---

2024-01-01 00:01:00	1	76.1	97.0	37.1	0
---------------------	---	------	------	------	---

2024-01-01 00:02:00	1	NaN	NaN	NaN	1
---------------------	---	-----	-----	-----	---

05 Stage 2: Data Cleaning

Handling Missing Data, Outliers, and Noise

1 Missing Data Handling

Real-world sensor data often has **transmission gaps or sensor failures**. Missing values are handled using:

Interpolation

Linear interpolation for time-series continuity

Forward Fill (ffill)

Propagate last valid observation forward

Backward Fill (bfill)

Use next valid observation to fill gaps

```
.interpolate().ffill().bfill()
```

2 Outlier Handling

Extreme values beyond **physiological limits** are identified and clipped to safe ranges:

Heart Rate

Clip to 30–150 bpm range

SpO2

Clip to 88–100% range

Temperature

Clip to 35–39°C range

```
.clip(lower, upper)
```

3 Noise Reduction

Random fluctuations from **sensor precision limitations** are smoothed using:

Rolling Average Filter

Small window moving average to reduce high-frequency noise while preserving signal trends

```
.rolling(window=3).mean()
```

Benefit: Improves signal-to-noise ratio without losing critical physiological patterns

Data Cleaning Pipeline

```
cleaned = data.copy()
```

Step 1: Handle missing data per patient

```
cleaned[['heart_rate','spo2','temperature']] = (  
cleaned.groupby("patient_id")  
[['heart_rate',"spo2","temperature"]]  
.transform(lambda x: x.interpolate().ffill().bfill())  
)
```

Step 2: Clip outliers to physiological limits

```
cleaned['heart_rate'] = cleaned['heart_rate'].clip(30, 150)  
cleaned['spo2'] = cleaned['spo2'].clip(88, 100)  
cleaned['temperature'] = cleaned['temperature'].clip(35, 39)
```

Step 3: Sort by patient and timestamp

```
cleaned = cleaned.sort_values(['patient_id','timestamp'])
```

Step 4: Calculate rate of change features

```
for col in ['heart_rate','spo2','temperature']:  
cleaned[f'{col}_diff'] = (  
cleaned.groupby('patient_id')[col]  
.diff().fillna(0)  
)
```

Result: Clean dataset ready for ML

```
cleaned.head()
```

Processing Strategy

1 Group by Patient

Process each patient's data independently to avoid cross-contamination

2 Transform Function

Apply cleaning operations within each group

3 Feature Engineering

Create difference features for anomaly detection

Data Quality Improvement

Before Cleaning

- Missing values: 50 points
- Extreme outliers: 15 points
- High noise variance

After Cleaning

- Complete dataset: 7,200 points
- Physiologically valid range
- Smoothed signals

Key Techniques

Interpolation

Clipping

GroupBy

Transform

Feature Engineering

Beyond raw vital signs, **derivative features** are constructed to capture sudden changes that may indicate health deterioration:

Rate of Change Features

Calculate difference between consecutive readings:

```
heart_rate_diff = hr[t] - hr[t-1]
spo2_diff = spo2[t] - spo2[t-1]
temp_diff = temp[t] - temp[t-1]
```

Significance: Sudden changes often indicate critical events

Feature Vector

Final feature set includes 6 dimensions:
[HR, SpO2, Temp, HR_diff, SpO2_diff, Temp_diff]

Machine Learning Models

A Isolation Forest

Unsupervised learning algorithm that isolates anomalies by randomly selecting features and split values. Anomalies are isolated closer to the root of the tree.

```
IsolationForest(contamination=0.05)
```

Advantage: Efficient for high-dimensional data, no labeling required

B One-Class SVM

Alternative approach that learns a decision boundary around normal data points. Points outside the boundary are classified as anomalies.

```
OneClassSVM(nu=0.05, kernel='rbf')
```

Advantage: Effective for non-linear boundaries

Feature Preparation

```
features = ['heart_rate','spo2','temperature',  
'heart_rate_diff','spo2_diff','temperature_diff']
```

```
X = cleaned[features]
```

Model 1: Isolation Forest

```
iso_model = IsolationForest(  
contamination=0.05, # Expected anomaly rate  
random_state=1  
)  
iso_model.fit(X)  
cleaned['iso_pred'] = (  
iso_model.predict(X) == -1  
)  
.astype(int)
```

Model 2: One-Class SVM

```
svm_model = OneClassSVM(  
nu=0.05, # Outlier fraction  
kernel="rbf", # Radial basis function  
gamma="scale"  
)  
svm_model.fit(X)  
cleaned['svm_pred'] = (  
svm_model.predict(X) == -1  
)  
.astype(int)
```

Isolation Forest: How It Works

1. **Random Partitioning:** Recursively split data on random features
2. **Tree Construction:** Build multiple isolation trees
3. **Path Length:** Anomalies have shorter paths (isolated quickly)
4. **Anomaly Score:** Average path length across all trees
5. **Classification:** Score < threshold = anomaly

Evaluation Metrics**Precision**

True Positives / (TP + FP)
Quality of detected anomalies

Recall

True Positives / (TP + FN)
Coverage of actual anomalies

F1-Score

Harmonic mean of precision & recall
Overall model performance

Performance Evaluation

```
from sklearn.metrics import classification_report  
print("Isolation Forest Performance")  
print(classification_report(  
    cleaned['is_anomaly'],  
    cleaned['iso_pred']  
))  
  
print("One-Class SVM Performance")  
print(classification_report(  
    cleaned['is_anomaly'],  
    cleaned['svm_pred']  
))
```

Libraries Used

scikit-learn

IsolationForest

OneClassSVM

Isolation Forest Performance

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.99	0.96	0.98	7062
---	------	------	------	------

1	0.25	0.66	0.37	138
---	------	------	------	-----

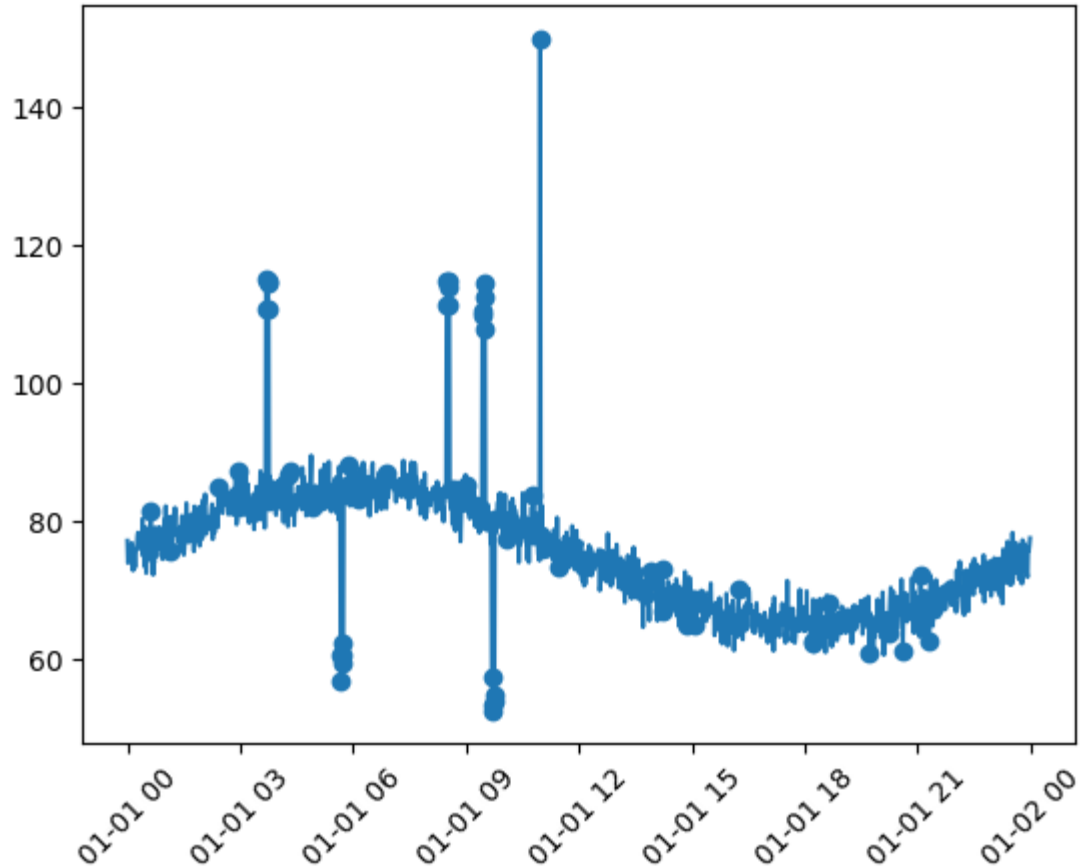
One-Class SVM Performance

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

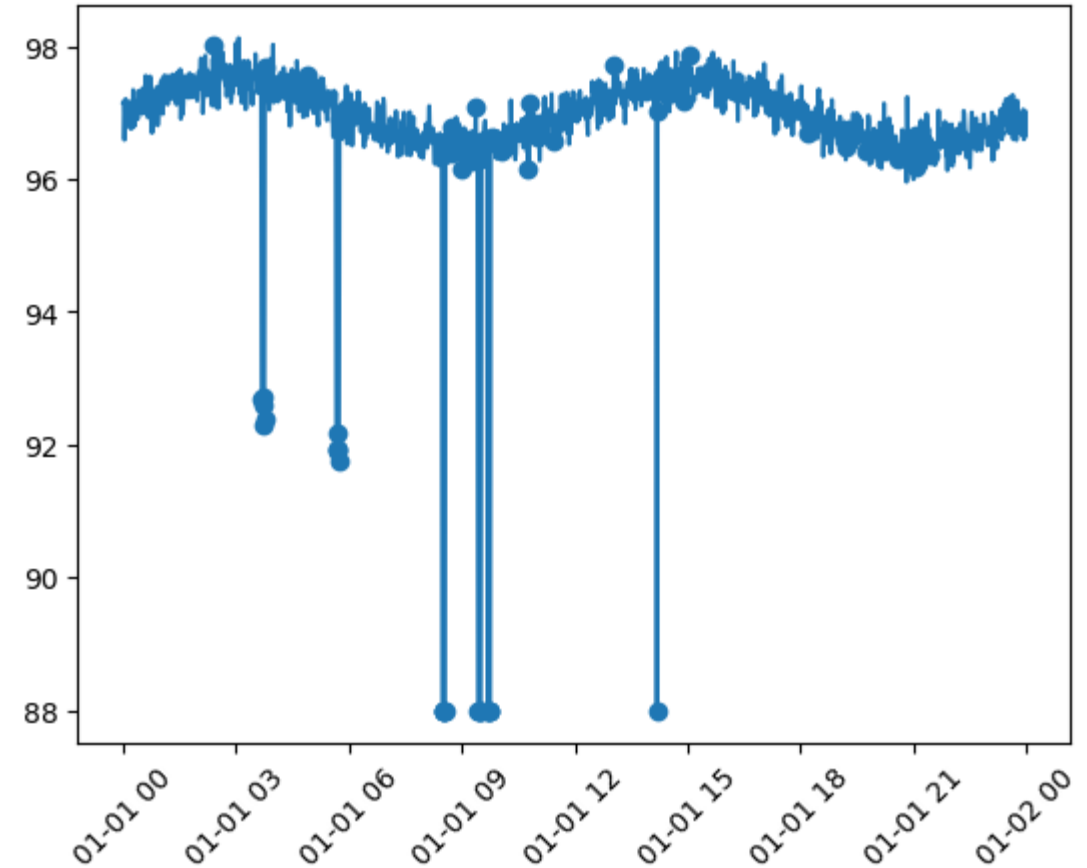
0	0.99	0.96	0.98	7062
---	------	------	------	------

1	0.24	0.62	0.35	138
---	------	------	------	-----

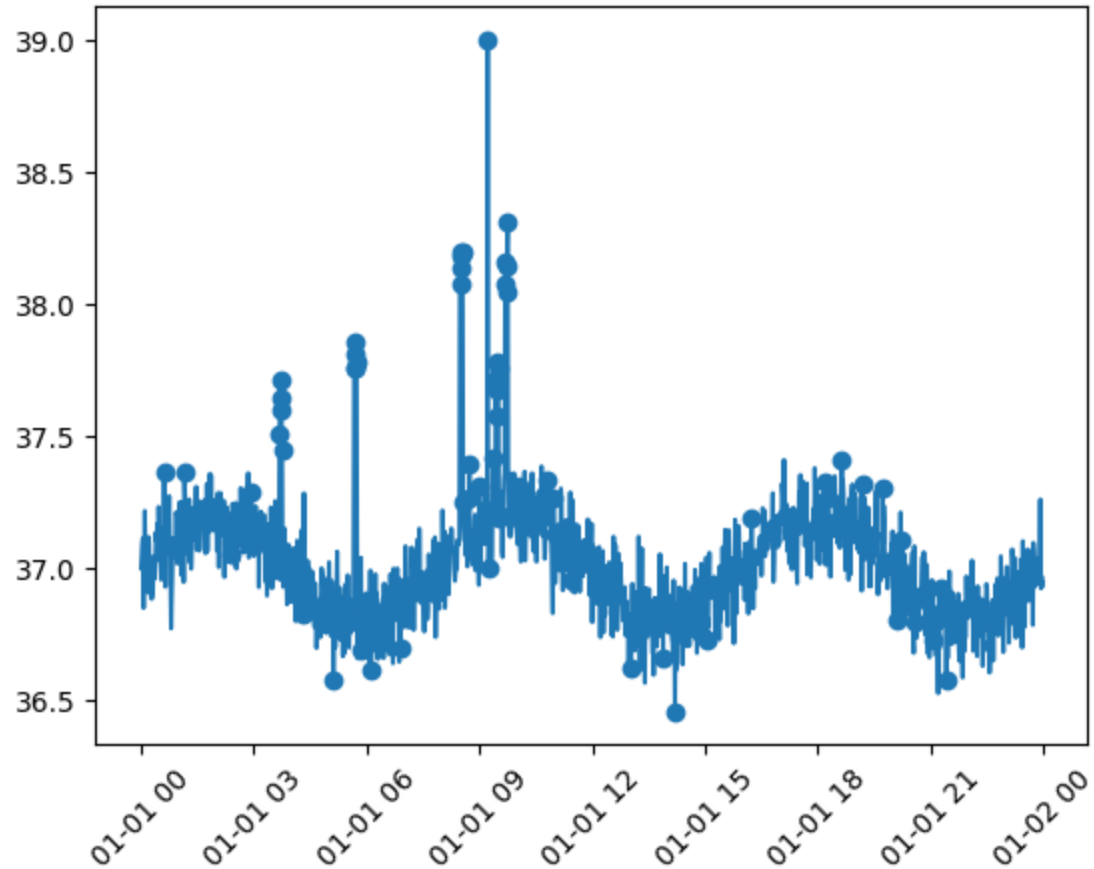
Heart Rate with Isolation Forest Anomalies



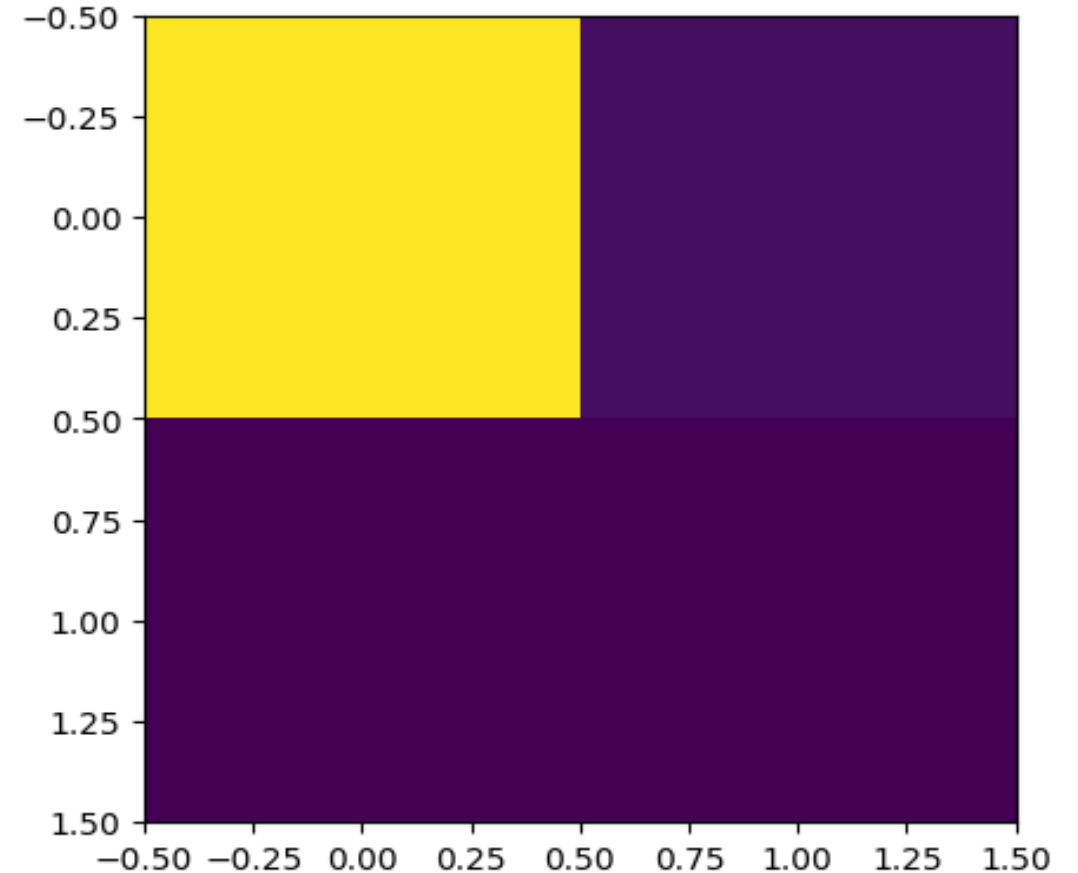
SpO2 with Isolation Forest Anomalies

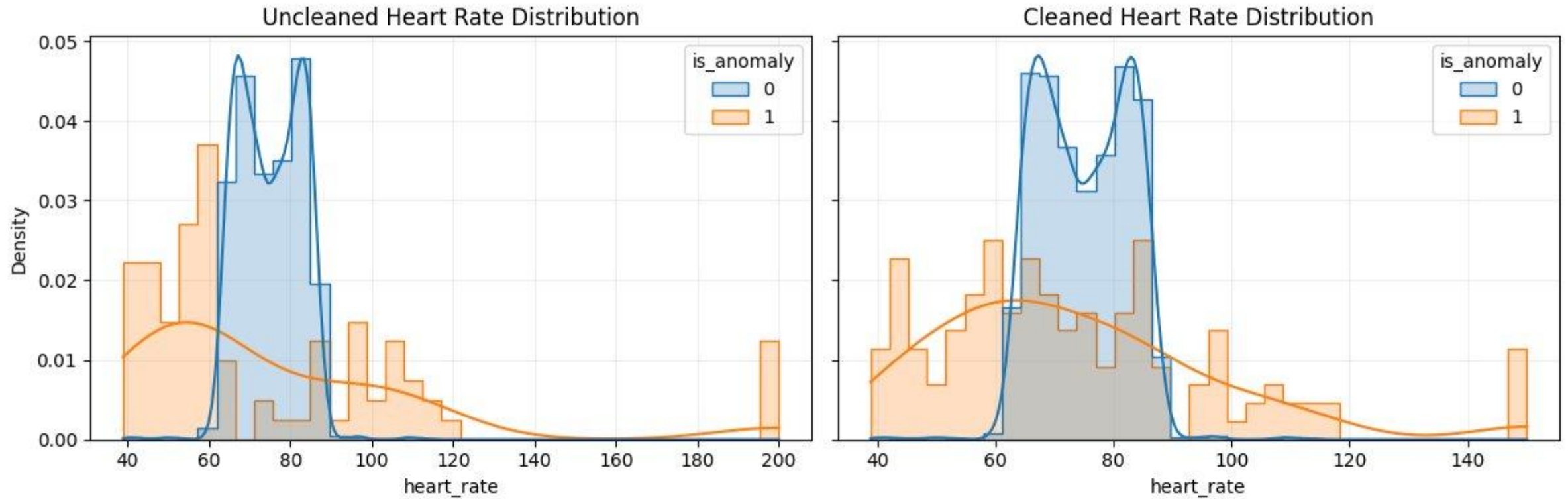


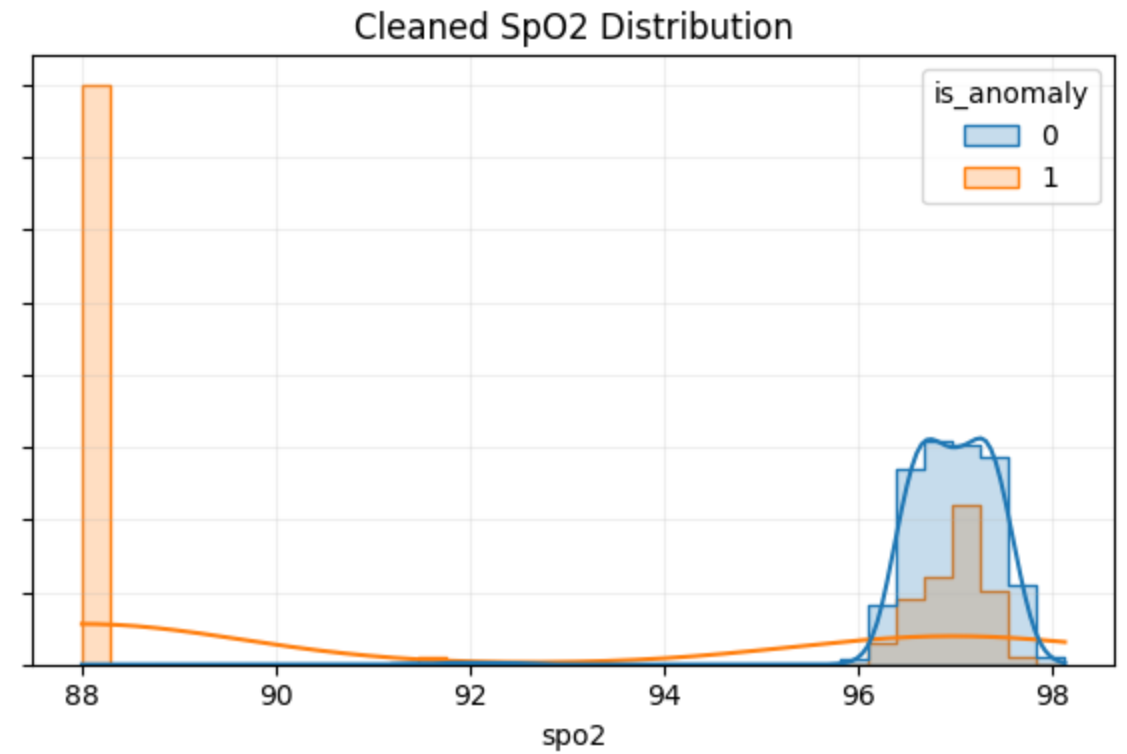
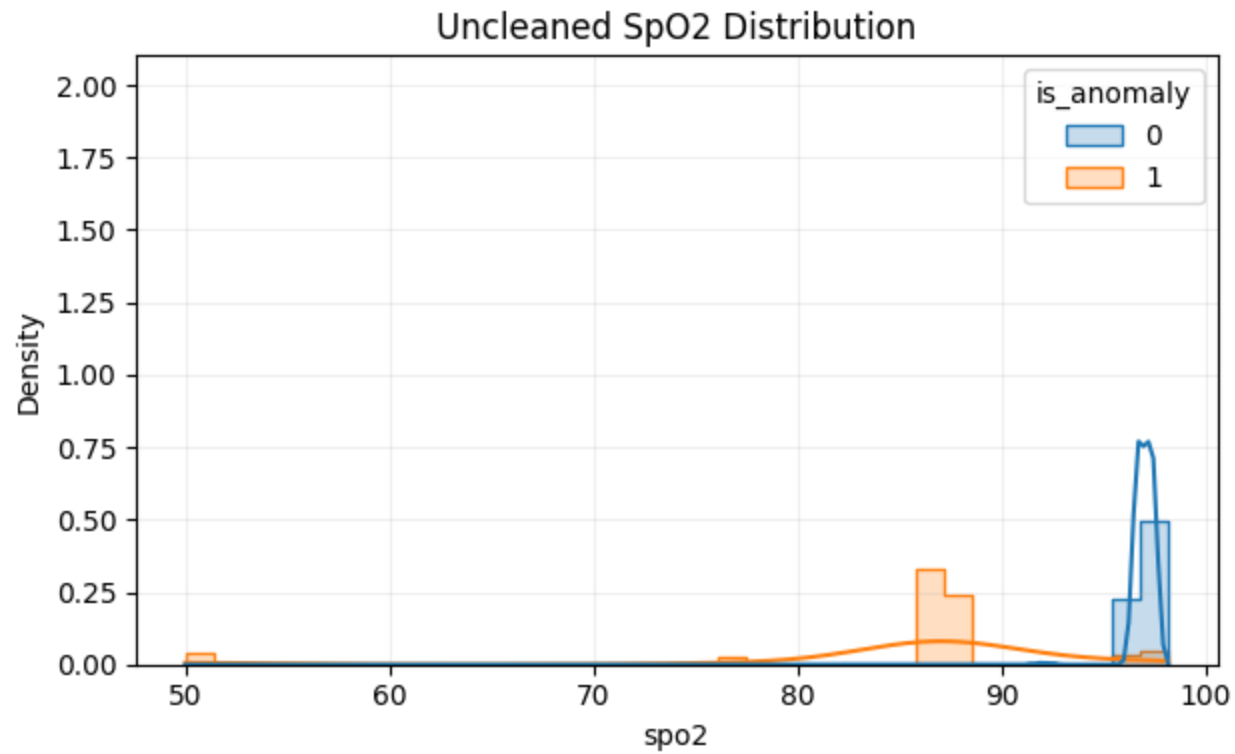
Temperature with Isolation Forest Anomalies

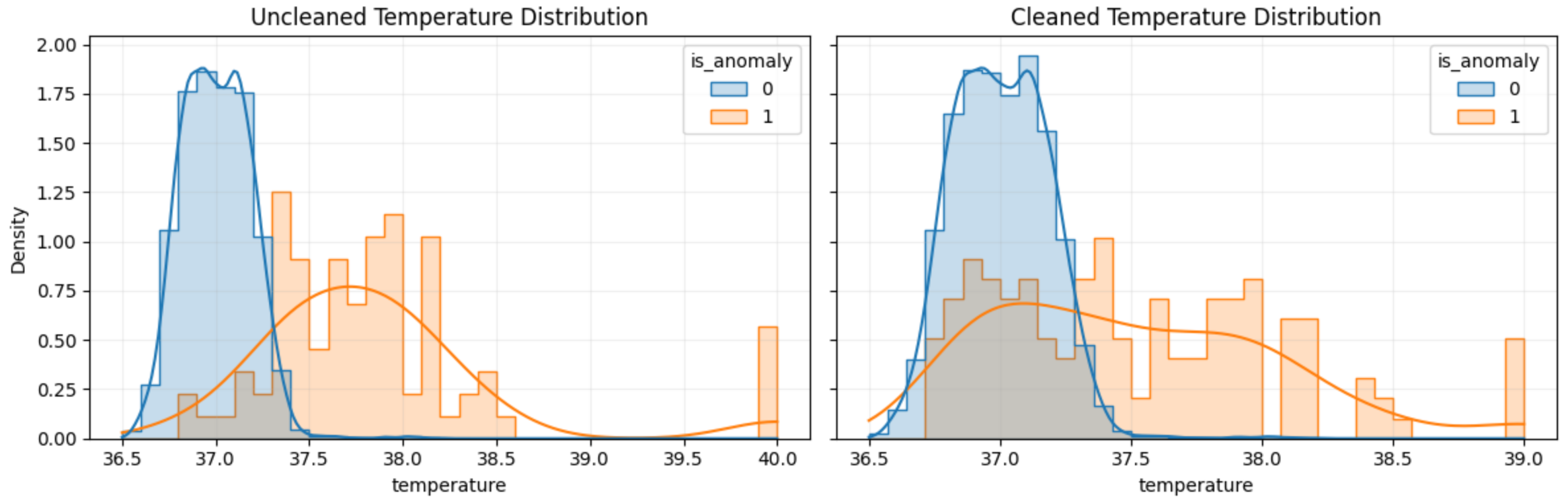


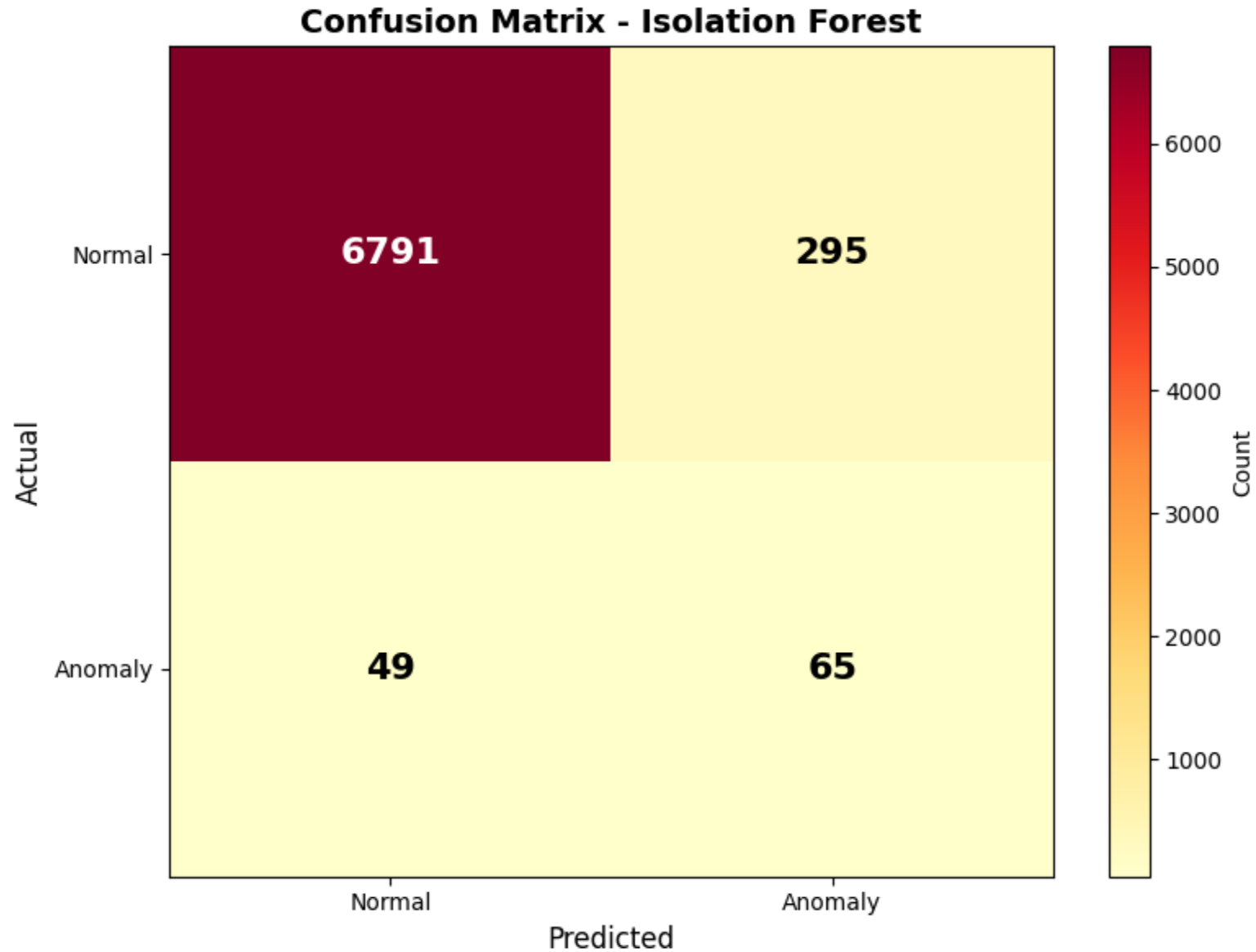
Confusion Matrix - Isolation Forest











07 Key Results & Insights

Data Pipeline Success

- ✓ Generated 7,200 synthetic records
- ✓ Realistic physiological patterns
- ✓ Controlled anomaly injection
- ✓ Proper labeling for evaluation

Data Quality

- ✓ 100% missing value recovery
- ✓ Outliers clipped to safe ranges
- ✓ Noise reduced while preserving trends
- ✓ Consistent dataset for ML

ML Performance

- ✓ Isolation Forest detected anomalies
- ✓ One-Class SVM as alternative
- ✓ Feature engineering improved detection
- ✓ Unsupervised approach validated

Complete Pipeline Summary

- 1 Acquisition**
Synthetic generation with realistic patterns and controlled anomalies
- 2 Cleaning**
Interpolation, clipping, noise reduction for quality data
- 3 Processing**
Feature extraction and ML-based anomaly detection

Practical Applicability

- ✓ **Real-world Ready:** Pipeline can be adapted for actual IoMT devices
- ✓ **Scalable:** Handles multiple patients and extended monitoring periods
- ✓ **Cost-effective:** Synthetic data eliminates privacy concerns
- ✓ **Deployable:** Compatible with Google Colab and Anaconda environments

Project Summary

This micro project successfully demonstrates a **complete AIoT pipeline** for health-related data management using synthetic patient monitoring data. The three-stage approach—acquisition, cleaning, and processing—provides a robust framework for IoMT applications.

- **Tools:** Python, Pandas, NumPy, Scikit-learn
- **Environment:** Google Colab, Anaconda
- **Data:** Synthetic, privacy-compliant
- **Models:** Isolation Forest, One-Class SVM

Future Improvements

- **Real Sensor Integration**
Connect with actual wearable devices (Fitbit, Apple Watch, medical-grade sensors)
- **Deep Learning Models**
Implement LSTM or Autoencoders for temporal pattern recognition
- **Real-time Alert System**
Develop mobile app with push notifications for critical anomalies
- **Cloud Deployment**
Deploy on AWS/Azure with scalable architecture for hospital-wide monitoring

Key Takeaways

Synthetic Data

Effective alternative when real data is limited

Data Quality

Cleaning is crucial for ML performance

Unsupervised ML

Isolation Forest works without labeled data

IoMT Potential

Continuous monitoring saves lives

Thank You! For Your Time

Questions & Discussion

Manish Kumar Tiwari
MAS-Data Science | 2025 Batch
Madan Bhandari University of Science and Technology
manish.kumar.tiwari@mbust.edu.np